

EdLearn — Full Project Audit vs. Literature Survey

2026-08-10

Contents

EDLEARN PROJECT STATUS	2
1. Executive Summary	2
2. What EdLearn Is Supposed To Be [DOC]	3
3. Current Implementation Status	3
4. Current Completion Percentage	9
5. Six Pedagogical Modes — Full Audit	11
6. RAG Audit	14
7. Ecosystem Gap Analysis (EdLearn → EdQuiz → EdCompass → EdMentor)	15
8. Personalization Gap	16
9. Engagement Gap	17
10. Cross-Domain Learning — the Survey's Named Differentiator	18
11. EdQuiz Gap Analysis	18
12. EdCompass Gap Analysis	19
13. EdMentor Gap Analysis	19
14. Peer Doubt-Solving	20
15. Audio / Voice Gap	20
16. Authentication & User Management	20
17. Security Audit	21
18. Testing Gap	21
19. Completed Features	22
20. Partially Completed Features	22
21. Broken Features	22
22. Mock / Placeholder Features	22
23. Feature Gaps (Master List)	22
24. Research-Driven Gaps From Literature Survey	25
25. Features That Must Be Added	26
26. MVP Features	26
27. P0/P1/P2/P3 Priority Backlog	27
28. 5-Member Responsibility Matrix	27
29. Detailed Member 1 Plan — AI Learning Engine	28
30. Detailed Member 2 Plan — RAG & Knowledge System	29
31. Detailed Member 3 Plan — Frontend & Learning Experience	29
32. Detailed Member 4 Plan — Backend, Database & Ecosystem	30
33. Detailed Member 5 Plan — Testing, DevOps, Security & Community	31
34. Dependencies	32
35. Parallel Work Plan	32
36. Integration Plan	32
37. Timeline	33
38. Git / Branch Strategy	34
39. Testing Plan	34
40. Security Plan	34
41. Deployment Plan	35
42. Final Definition of Done	35

EDLEARN PROJECT STATUS

Complete Codebase Audit vs. the EdLearn Literature Survey — Gap Analysis & 5-Member Execution Plan

Audit date: 2026-08-10. Reference document: *“Literature Survey — EdLearn: The AI-Powered Learning Ecosystem”* (Development Team, July 2026), supplied by the user. Codebase: feature/ui-improvements branch, working tree as of this session.

Labeling convention used throughout: [CODE] = directly verified by reading the source; [DOC] = a claim/vision statement taken from the literature survey; [INFERENCE] = a recommendation or judgment call not directly stated in either source.

1. Executive Summary

The literature survey’s entire thesis is that EdLearn’s differentiator is **six AI-driven pedagogical modes, several of which withhold direct answers**, grounded in RAG, feeding a connected ecosystem (EdQuiz/EdCompass/EdMentor). Having now read that document and re-examined the codebase specifically against it, the single most important finding of this audit is this:

[CODE] The shipped frontend has explicitly disabled all six pedagogical modes. In `LeftNavigationPanel.tsx`, the mode-selector UI defines all six modes (Accelerator, Socratic Practice, Concept Simplifier, Personalized Roadmap, Gap Finder, Feynman Test) in an array, but renders none of them as clickable buttons — only a static line of copy is shown: *“Modes have been suspended to focus entirely on high-quality study notes.”* The only mode a real user can currently select is **Duo Podcast** (mode 7) — a Sprint-4 addition that doesn’t even appear in the literature survey’s six modes.

[CODE] Even where the backend still defines mode-specific prompts (`pedagogicalEngine.ts`), the main `/api/generate` route never calls them for modes 1, 2, 3, 5, or 6. It uses one fixed “generate textbook study notes” prompt and one fixed output schema (`title/introduction/outline/contentBlocks`) regardless of which mode number is sent. The Socratic mode’s `scenario/trapOptions`, the Simplifier’s `keystoneQuestions`, the Gap Finder’s `questions[]/trapIndex`, and the Feynman mode’s `promptQuestion/jargonChecklist` schemas are fully written in `pedagogicalEngine.ts` — and never invoked by this route. `LivingDocument.tsx`, the component that renders generated content, has no rendering branch for any of those schemas either — only the generic notes schema and the podcast script. **The six modes are real, well-written prompt designs sitting in dead code, not a working feature.**

This changes the honest answer to “does EdLearn do what the literature survey says it does?” from “partially” to: **EdLearn currently ships one working pedagogical experience** (AI-generated study notes, genuinely grounded via live web retrieval, genuinely double-reviewed by a second LLM pass) **plus one bonus feature not in the original research scope** (Duo Podcast). Everything the survey identifies as the actual differentiator — answer-withholding Socratic dialogue, adversarial Feynman testing, and above all **Cross-Domain Learning, which the survey calls EdLearn’s one truly unmatched idea** — is either disconnected dead code or, in the case of Cross-Domain Learning, a UI toggle that computes a prompt string and then never sends it anywhere (confirmed in `InteractiveAssistant.tsx`: the `systemPrompt` variable built for the “Cross-Domain” chat toggle is never included in the actual API call).

The ecosystem vision (shared context flowing into EdQuiz, EdCompass, EdMentor) is likewise not implemented beyond a one-way SSO redirect — no data flows back from any sibling app, confirmed by the Hub page’s own code comment: *“EdMentor/EdCompass haven’t confirmed what they can share back with EdLearn yet... the flags below stay hardcoded to false.”*

None of this means the codebase is bad — the parts that do work (auth, the notes-generation pipeline itself, Vision Board, badges/progress, RAG-lite grounding, IDOR discipline) are solid engineering. But a

reviewer, investor, or professor evaluating EdLearn against this literature survey today would find the survey describing a product that is, in its most distinctive respects, not yet built.

2. What EdLearn Is Supposed To Be [D0C]

Per the literature survey: EdLearn is a self-learning app teaching through **six AI-driven pedagogical modes** — several deliberately withholding direct answers in favor of guided questioning — grounded in real source material via RAG, and connected to three sibling tools: **EdQuiz** (assessment), **EdCompass** (career roadmap), and **EdMentor** (human mentorship).

The survey’s four identified market gaps, and EdLearn’s proposed answer to each:

Market gap [D0C]	EdLearn’s proposed contribution [D0C]
No competitor combines rigorous answer-withholding teaching with an integrated assessment → roadmap → mentorship pipeline	Route every module through shared context into EdQuiz, EdCompass, EdMentor
No competitor tracks an individual learner’s changing skill relevance (only done at national policy level today — China/Japan/Singapore)	Personalized Roadmap + Knowledge Gap Finder modes as a learner-facing skill-relevance signal
Rigor and engagement trade off in every reviewed product (Khanmigo rigorous-but-unused at 15% adoption; Duolingo Max engaging-but-shallow)	Socratic + Feynman modes for rigor, paired with proactive, in-workflow design to fix the engagement problem
Cross-domain/lateral-thinking instruction has no equivalent anywhere in the market	Cross-Domain Learning mode — the survey explicitly calls this EdLearn’s “unmatched” differentiator

The survey also flags a specific technical caution [D0C]: Physics Wallah needed 150,000 curated reasoning traces on top of RAG to get multi-step math right — **RAG alone grounds facts, not reasoning**, which is directly relevant to how EdLearn’s own retrieval pipeline should be evaluated (see Section 3.5).

And a specific product-adoption caution [D0C]: South Korea’s mandatory-AI-textbook rollback after 4 months is cited as a warning to **keep EdLearn opt-in, not forced** — relevant context for however aggressively engagement/gamification features get built later.

3. Current Implementation Status

3.1 Feature Inventory

Feature	Intended by Reference	Frontend	Backend	Database	AI	Integration	Tested	Current Status
Email/password + Google OAuth auth	Implied (auth deferred to “simple email/password” per survey §9.2)	[OK]	[OK]	[OK]	—	Google OAuth code-complete, no live creds in this repo’s .env	[UNTESTED]	[DONE] COM- PLETE (email/password); [UNTESTED] NOT TESTED (Google OAuth — unverifi-able without live credentials)
AI-generated study notes (single generic pipeline)	Implied as the base of the 6-mode system	[OK]	[OK]	[OK]	[OK] real (Groq/Gemini, 2-pass review)	—	[UNTESTED]	[DONE] COM- PLETE — this is the one fully working AI peda- gogical feature in the product
6 distinct peda- gogical modes (Accelerator, Socratic, Simplifier, Roadmap, Gap Finder, Feynman)	Core thesis of the survey	[BROKEN] defined, dis-abled in UI	[BROKEN] prompts defined, never called by /api/generate for 5 of 6 modes	—	[BROKEN] real prompts exist, unreachable	—	[UNTESTED]	[BROKEN] BRO- KEN / EFFEC- TIVELY NOT IMPLE- MENTED — see Section 5 for the full mode-by-mode trace

Feature	Intended by Reference	Frontend	Backend	Database	AI	Integration	Tested	Current Status
Answer-withholding / Socratic guided questioning	Core thesis of the survey , backed by cited re-search (Khan-migo, Coursera Coach data)	[MISSING] not reachable	[PARTIAL] — prompt exists, not wired to a live route		[PARTIAL] —		[MISSING]	[MISSING] NOT IMPLEMENTED IN THE SHIPPED PRODUCT despite a complete, well-written prompt existing in pedagogicalEngine.ts
Feynman adversarial testing	Core thesis	[MISSING]	[PARTIAL] — (same as above)		[PARTIAL] —		[MISSING]	[MISSING] NOT IMPLEMENTED IN THE SHIPPED PRODUCT
Cross-Domain / lateral learning	The survey's single named differentiator (“no equivalent anywhere in the market”)	[MOCK] dead toggle	[MISSING] — no back-end mode exists at all		[MISSING] —		[MISSING]	[MOCK] MOCK/PLACEHOLDER worst state in the app — see Section 10
RAG-grounded generation	Core thesis, with an explicit caution about reasoning-heavy tasks	[OK] (citations rendered)	[OK]	[OK] (Citation table)	[PARTIAL] — real retrieval, no embeddings		[UNTESTED]	[PARTIAL] PARTIAL — see Section 6 for the full RAG audit

Feature	Intended by Reference	Frontend	Backend	Database	AI	Integration	Tested	Current Status
Duo Podcast (2-voice audio dialogue)	Not in the survey's scope at all — an EdLearn team addition beyond the research brief	[OK]	[OK]	[OK]	[OK] real	—	[UNTESTED]	[DONE] COMPLETE , though built on an unofficial, keyless TTS end-point (see Section 15)
EdQuiz integration	Core ecosystem thesis	[OK] (SSO redirect button)	[OK] (mint-side handoff + intent classifier)	—	[OK] (real classification call)	[MOCK] one-way redirect only	[MISSING]	[MOCK] MOCK/PLACEHOLDER at the integration level — see Section 11
EdCompass integration	Core ecosystem thesis	[OK] (SSO redirect + Hub card)	[OK] (same handoff mechanism)	—	—	[MOCK] one-way redirect, hard-coded false for real data	[MISSING]	[MOCK] MOCK/PLACEHOLDER — see Section 12
EdMentor integration	Core ecosystem thesis	[OK] (SSO redirect + Hub card)	[OK] (same handoff mechanism)	—	—	[MOCK] one-way redirect, hard-coded false for real data	[MISSING]	[MOCK] MOCK/PLACEHOLDER — see Section 13
Personalization (skill-relevance signal)	Core ecosystem thesis	[PARTIAL] basic profile only	[PARTIAL] basic profile only	[PARTIAL] (UserProfile, goal, skills, time, difficulty)	[MISSING] —	—	[MISSING]	[MISSING] NOT IMPLEMENTED beyond a static onboarding profile — see Section 8

Feature	Intended by Reference	Frontend	Backend	Database	AI	Integration	Tested	Current Status
Engagement (streaks, goals, notifications, proactive AI)	Explicitly identified research gap (Khan-migo's 15% usage problem)	[MISSING]	[MISSING]	[MISSING]	[MISSING]	—	[MISSING]	[MISSING] NOT IMPLEMENTED — see Section 9
Peer doubt-solving forum	Listed as a “planned capability” [DOC]	[MISSING]	[MISSING]	[MISSING]	—	[MISSING]	[MISSING]	[MISSING] NOT IMPLEMENTED (also documented as previously built in context.md but absent from the current code — a doc/code mismatch independent of this survey)

Feature	Intended by Reference	Frontend	Backend	Database	AI	Integration	Tested	Current Status
Audio (TTS)	Listed as “planned” [DOC]	[OK]	[OK]	[OK]	n/a	—	[UNTESTED]	[DONE] AHEAD OF THE SURVEY’S EXPECTATION — TTS already exists (fragile dependency, see Section 15)
Voice conversation / STT	Not explicitly promised, but implied by “AI voice tutor” framing	[MISSING]	[MISSING]	—	—	—	[MISSING]	[MISSING] NOT IMPLEMENTED
Vision Board (goals CRUD)	Not mentioned in the survey at all	[OK]	[OK]	[OK]	—	—	[MISSING]	[DONE] COMPLETE — an EdLearn team addition beyond the research brief, and the most solid feature in the whole codebase

Feature	Intended by Reference	Frontend	Backend	Database	AI	Integration	Tested	Current Status
Market-Demand trending-skills widget	Loosely related to the “skill-relevance” gap, but not the individual-learner version the survey describes	[OK]	[OK]	[OK] (Mongo)	—	—	[MISSING]	[MOCK] HALF MOCK — GitHub half is real, “LinkedIn” half is a hard-coded array with random jitter
Automated testing	Not addressed by the survey	[MISSING]	[MISSING]	—	—	—	[MISSING]	[MISSING] NOT IMPLEMENTED — zero test files anywhere in the repository

3.2 Infrastructure snapshot [CODE] (carried forward from the prior full-stack audit, still current)

- CORS is effectively open (origin: true ignores the computed allowlist).
- Rate limiting is declared as a dependency but applied nowhere.
- Kubernetes manifests will not boot the backend as configured — required secrets (JWT_SECRET, DATABASE_URL, etc.) are never injected, only NODE_ENV/PORT/Frontend_URL are in the ConfigMap.
- .env files are correctly gitignored and not committed — a genuine strength.
- No automated tests exist anywhere in either package.

4. Current Completion Percentage

Overall: ~41%, scored specifically against *this* reference document’s definition of “done” — which is a stricter bar than a generic feature checklist, because the survey’s whole value proposition rests on the pedagogical modes and cross-domain learning actually working, and they currently don’t reach the end user.

Category	%	Basis
Frontend	65%	Every core page renders and wires to a real API, but the mode selector — the product’s namesake feature — is switched off, and the AI Tutor chat is non-functional as a chat (carried from prior audit).
Backend	60%	Auth, notes-generation, dashboard, vision-board, TTS routes are solid; the pedagogical-mode routing logic is the one glaring architectural gap.
Database	75%	Schema is coherent; nothing in the schema tracks per-topic mastery, knowledge gaps, or skill relevance over time — the exact signal the survey says is EdLearn’s second differentiator.
AI	55%	The core generation pipeline (real Groq/Gemini calls, double-pass review) is genuinely good work. Scored down hard because the <i>specific</i> AI behaviors the survey is built around — withholding answers, Socratic questioning, adversarial Feynman testing, cross-domain synthesis — are not reachable in the live product.
RAG	35%	Real retrieval exists (Wikipedia search + optional URL scrape, injected into the prompt with citation instructions) — this is genuine grounding, not fake. But there is no chunking, no embeddings, no vector store, no similarity search, no user-uploaded documents, and no retrieval-quality evaluation. See Section 6.
Authentication	75%	Solid password hashing, JWT, IDOR discipline; missing password-reset, email verification, and (per the survey’s own note) OAuth was meant to be deferred for the first prototype in favor of simple email/password, which is in fact the more complete path here.

Category	%	Basis
EdQuiz	15%	Only a labeled redirect button and an intent classifier exist; no quiz generation, no score-based feedback loop into EdLearn.
EdCompass	10%	Only a labeled redirect button and a Hub card gated behind a hardcoded false; no shared data at all.
EdMentor	10%	Same as EdCompass.
Personalization	20%	A static onboarding profile (goal, skills, available time, difficulty) exists and is used to seed the roadmap prompt once; nothing tracks mastery, gaps, or changing skill relevance afterward.
Cross-Domain Learning	5%	A UI toggle whose computed prompt is never sent to any API. This is the survey's headline differentiator and the least-built feature in the repository.
Testing	0%	No unit, integration, or E2E tests exist anywhere.
Security	45%	Strong password hashing and IDOR checks; CORS is effectively disabled and rate limiting is absent — both documented elsewhere as “fixed” but not present in the running code.
Deployment	40%	Docker Compose is coherent; Kubernetes manifests exist but won't boot the backend as configured (missing Secrets).

Methodology [INFERENCE]: each category was scored by asking “if a user/reviewer tried to actually exercise this capability today, using only the shipped UI and the code that runs behind it, would it work as the reference document describes?” — not by counting files, routes, or UI components that merely exist. This is why RAG, AI, and Cross-Domain Learning score much lower here than a naive “does the code exist” count would suggest: the code for several pieces exists but is disconnected from anything a user can reach.

5. Six Pedagogical Modes — Full Audit

Pedagogical Mode	Current Implementation [CODE]	Missing	Priority
Mode 1 — Accelerator (Pareto 80/20)	Prompt defined in <code>pedagogicalEngine.ts</code> (schema: <code>keystoneConcepts</code> , <code>beginnerPitfalls</code> , <code>acceleratedBuild</code>). Not called by <code>/api/generate</code> — that route always uses the generic notes prompt regardless of mode number. UI button for this mode does not render (modes “suspended”).	Wiring <code>/api/generate</code> ’s mode-1 branch to actually call <code>getPedagogicalModeConfig(1,...)</code> ; a frontend renderer for the <code>keystoneConcepts/acceleratedBuild</code> schema; re-enabling the mode-selector UI.	P0
Mode 2 — Socratic Practice	Prompt defined (schema: <code>scenario</code> , <code>question</code> , <code>trapOptions</code> , <code>correctAnswer</code> , <code>guidingHint</code>) — this is the mode the entire survey’s “answer-withholding” thesis rests on. Same disconnection as Mode 1: never called, no UI entry point, no renderer. Does not currently ask guiding questions or withhold answers in the live product — a user selecting “Socratic” (if they even could) would receive the exact same generic notes JSON as any other mode.	Same three items as Mode 1, plus: an interactive UI for presenting the scenario/trap-options/hint flow (a static notes renderer cannot display this schema at all today).	P0 — this is the survey’s single most load-bearing claim
Mode 3 — Concept Simplifier (physical analogy)	Prompt defined (schema: <code>analogyTitle</code> , <code>explanation</code> , <code>keystoneQuestions[]</code> with <code>correctIndex</code>). Same disconnection.	Same three items; a quiz-style renderer for <code>keystoneQuestions</code> .	P1

Pedagogical Mode	Current Implementation [CODE]	Missing	Priority
Mode 4 — Personalized Roadmap	The only one of the six that is actually wired up — but only via the separate <code>/api/roadmap</code> route (hardcoded to always call mode 4), not as a selectable in-workspace “mode.” So the roadmap-generation flow works end-to-end; treating it as one of “six selectable pedagogical modes” inside the study workspace is inaccurate to what the code does.	Nothing functionally — it works — but it should probably be reclassified as “the onboarding roadmap generator,” not a workspace pedagogical mode, to match what it actually is.	P2 (documentation/framing fix, not a build task)
Mode 5 — Gap Finder	Prompt defined (schema: 5 MCQs with <code>trapIndex</code> designed to expose superficial understanding). Same disconnection as Modes 1-3: never called, no UI entry point, no renderer, and critically no persistence — even if wired up, nothing in the schema writes “this learner has a gap in X” anywhere the roadmap or dashboard could read it back.	Wiring + renderer (as above) plus a <code>KnowledgeGap</code> or similar table so a detected gap actually feeds back into personalization (see Section 8) — without this, Gap Finder is a one-off quiz with no memory.	P0 — this is the survey’s second named differentiator (individual skill-relevance signal)
Mode 6 — Feynman Test	Prompt defined (schema: <code>promptQuestion</code> , <code>jargonChecklist</code> — an adversarial “explain it to a 10-year-old” listener). Same disconnection.	Same three items as Mode 1, plus a genuinely interactive multi-turn UI (the current chat-shaped UI that exists, <code>InteractiveAssistant.tsx</code> , is broken for exactly this kind of back-and-forth — see Section 3.1 of the prior audit).	P1

Pedagogical Mode	Current Implementation [CODE]	Missing	Priority
(Bonus) Mode 7 — Duo Podcast	Fully implemented and working end-to-end, including its own prompt, TTS synthesis, and a dedicated PodcastPlayer UI. Not part of the survey's six modes at all.	Nothing critical — this genuinely works.	n/a (already shipped)

Overall verdict on this section: of six research-backed pedagogical modes, **zero are reachable by an end user today**, one (Roadmap) works but only outside the mode-selector concept, and the only mode a user can actually pick in the workspace is a bonus feature (Podcast) the research document never asked for. This is, by a wide margin, the highest-priority engineering gap in the entire project relative to the stated research goals.

6. RAG Audit

Verdict: PARTIAL — real grounding, but not the retrieval architecture the survey implies, and no path yet to the reasoning safeguards the survey explicitly recommends.

Capability	Status [CODE]
Document ingestion	[MISSING] Not implemented — no user-uploaded PDFs/notes/files are ever ingested for RAG.
Document parsing / chunking	[MISSING] Not implemented — the scraper (<code>lib/scrapper.ts</code>) takes a Wikipedia extract or a single scraped page's paragraph text as one undivided block (capped at ~10,000 characters), not chunked units.
Embeddings	[MISSING] Not implemented — no embedding model call anywhere in the codebase (confirmed by a full-repo search for <code>embedding/vector/pinecone/pgvector/chroma/faiss</code> — zero matches).
Vector storage	[MISSING] Not implemented — no vector database of any kind is configured or referenced.
Similarity search / retrieval	[PARTIAL] A simpler substitute exists: Wikipedia's own keyword search API returns the top 2 matching articles, or a single user-supplied URL is scraped directly. This is real retrieval, just not semantic/vector retrieval — closer to "keyword search + stuff into prompt" than RAG in the embedding sense.

Capability	Status [CODE]
Context injection	[DONE] Real — retrieved content is genuinely formatted into numbered [Source N] blocks and injected into the system prompt for both the notes-generation and podcast-generation prompts.
Source citations	[DONE] Real — the model is instructed to cite [1]/[2] inline, and citations are persisted to a dedicated Citation table linked to each Topic. This is a genuine strength.
Grounded responses	[PARTIAL] Grounded against whatever the top-2 Wikipedia search results (or one arbitrary user URL) happen to contain — reasonable for general-knowledge topics, unverified for anything requiring authoritative or current sources.
User-uploaded learning material	[MISSING] Not implemented.
Personal notes as RAG source	[MISSING] Not implemented.
Personal knowledge base	[MISSING] Not implemented.
Retrieval evaluation	[MISSING] Not implemented — no mechanism checks whether the retrieved Wikipedia/URL content is actually relevant before it's used.

Reasoning-safeguard check, specifically called for by the survey [DOC]: the survey explicitly warns that Physics Wallah needed 150,000 curated reasoning traces beyond RAG to get multi-step math right, and recommends EdLearn's pipeline "plan for this once it goes beyond simple explanations." **[CODE]** EdLearn's current double-pass review (/api/generate's second LLM call) audits citation placement, Mermaid-diagram syntax, and factual alignment against the *same* retrieved sources — it does not independently re-derive or verify multi-step reasoning (e.g., a math derivation or code-logic proof). For STEM/code-heavy topics, this means the existing "double-pass review" is a grounding/style check, not the reasoning-verification safeguard the survey specifically recommends. **[INFERENCE]** This is worth flagging as a distinct future workstream, not solved by the current double-pass step.

****What's needed for production-ready RAG [INFERENCE, priority-ordered]:**** 1. A real chunking + embedding pipeline (even a lightweight one — e.g., pgvector on the existing Postgres instance, given the survey itself flags pgvector vs. Pinecone as exactly the open question to research next [DOC]). 2. User-uploaded document ingestion (PDF/notes) — directly requested by the survey's own "personal knowledge base" framing and comparable to StudyFetch's architecture, which the survey calls out as "architecturally close to EdLearn's own RAG-on-personal-notes plan" [DOC]. 3. A basic retrieval-relevance check before injecting scraped content into a prompt (cheap: a similarity/keyword-overlap threshold; not necessarily a full reranker at this stage). 4. A separate reasoning-verification step for STEM/code content, distinct from the existing citation/style double-pass review.

7. Ecosystem Gap Analysis (EdLearn → EdQuiz → EdCompass → EdMentor)

[CODE] Confirmed: these are one-way redirect links with a shared login token, not a data-sharing ecosystem.

Question	Answer [CODE]
Does EdLearn connect to EdQuiz?	Only via /api/sso/handoff minting a 90-second signed JWT and redirecting the browser to <code>edquiz.theedmentor.com/sso?....</code>

Question	Answer [CODE]
Does quiz performance affect learning?	No — there is no route or table anywhere in this repo that receives quiz results back from EdQuiz.
Does EdLearn connect to EdCompass?	Same SSO redirect mechanism only.
Do knowledge gaps affect career roadmap recommendations?	No — and this can't currently happen even in principle, because (per Section 5) the Gap Finder mode that would detect a knowledge gap isn't reachable, and even if it were, nothing persists its results.
Does EdLearn connect to EdMentor?	Same SSO redirect mechanism only.
Can mentors see learner progress?	No — confirmed by the Hub page's own code comment: mentor/compass data flags are hardcoded false because "EdMentor/EdCompass haven't confirmed what they can share back with EdLearn yet (no cross-app access/credentials)."
Is there shared learner context?	No.
Is there a unified learner profile?	No — UserProfile exists only within EdLearn's own database.
Is there cross-product authentication?	Partially — the SSO handoff genuinely avoids a second login prompt on the receiving app (a real, well-built piece of infrastructure), but that's authentication continuity, not shared context.
Is data synchronized?	No, in either direction.
Are these real integrations or just links/buttons?	Links/buttons with a signed token attached. The minting side (this repo) is well-engineered (separate secret, short TTL, topic-passing for EdQuiz). The receiving side is explicitly out of this repo's scope per its own code comments, and no sibling app currently sends anything back.

Missing pieces [INFERENCE], in dependency order: (1) a defined API contract each sibling app must implement to push results back to EdLearn (quiz scores, career-fit results, mentor session logs); (2) a shared learner-context store or webhook receiver on EdLearn's side to accept that data; (3) the Hub page and dashboard wired to real data instead of the current hardcoded-false placeholders; (4) once #1-3 exist, closing the loop the survey specifically calls for — a Gap Finder result actually changing what EdCompass recommends.

8. Personalization Gap

[CODE] Current state: UserProfile stores fullName, careerGoal, currentSkills[], availableTime, difficulty — captured once at onboarding and editable via PATCH /api/profile. That's the entire personalization surface.

Signal the survey implies EdLearn should track [DOC]	Present? [CODE]
Learning history	[PARTIAL] Partial — Topic/Progress tables log what was generated and completed, but nothing summarizes it into a profile-level signal.
Skill profile / strengths / weaknesses	[MISSING] Not implemented.

Signal the survey implies EdLearn should track [DOC]	Present? [CODE]
Knowledge gaps	[MISSING] Not implemented (would depend on Gap Finder mode actually running — see Section 5).
Goals	[DONE] careerGoal field exists.
Preferred learning style	[MISSING] Not implemented.
Learning speed	[MISSING] Not implemented (availableTime is a static onboarding input, not a measured behavior).
Quiz performance	[MISSING] Not implemented — no quiz-scoring exists inside EdLearn itself.
AI conversation history / memory	[MISSING] Not implemented — every /api/generate call is stateless; there is no persisted conversation memory across turns or sessions.
Topic mastery	[MISSING] Not implemented — Progress/Topic.completed only record “read/scroll through,” not demonstrated mastery.
Recommended next topic	[MISSING] Not implemented — the roadmap, once generated, is static; nothing re-ranks or suggests what to study next based on performance.
Personalized roadmap	[PARTIAL] Generated once at creation time from careerGoal/difficulty/availableTime; never adapts afterward.
Skill relevance (the survey’s specific national-policy-level analogy)	[MISSING] Not implemented at the individual level — this is the exact gap the survey names as EdLearn’s second differentiator, and it doesn’t exist yet.

What’s needed [INFERENCE]: a SkillMastery/KnowledgeGap data model keyed to (user, topic); a real scoring mechanism (which requires Gap Finder or an equivalent quiz to actually run); a lightweight “next topic” recommender that reads that table; and a feedback loop so the roadmap can adapt rather than stay static after generation. This is a multi-week body of work, not a quick fix, and should be sequenced after the pedagogical-modes fix in Section 5 (there’s no gap signal to track until a mode that produces one is actually reachable).

9. Engagement Gap

[CODE] Confirmed absent, by direct search of the backend: no streaks, no daily-goal tracking, no notifications/reminders, no adaptive difficulty, and no proactive “here’s what to do today” push — confirmed by a repo-wide search for notification/reminder/streak/daily goal, which returns zero matches anywhere in backend/src.

What does exist: the Badge system (course-completion only, not per-topic or streak-based), the “Facts” widget (a static “did you know” feed tied loosely to the active roadmap topics — informative, not a learning plan), and Day-completion tracking.

[DOC] The survey’s own research is unambiguous on why this matters: Khan Academy’s Khan-migo, despite being pedagogically rigorous and having 108 million interactions since 2023, is used voluntarily by only 15% of eligible students — described in the survey as “a chat window people forget to open.” **[INFERENCE]** EdLearn’s current design has the same passive shape: a student has to remember to open the workspace and pick a day. There is no mechanism that brings the learner back or tells

them what to do today.

****Recommended additions, without compromising rigor [INFERENCE, per the brief's explicit instruction not to force adoption – see South Korea caution in Section 2]:**** - A daily-goal / "continue learning" surface on the dashboard, generated from wherever the learner left off (low effort – the data to compute this already exists inProgress/Roadmap). - A lightweight streak counter (low effort, additive to the existingProgresstable). - An expansion of the existing Facts feed into a genuine "today's suggested next step" – not gamified pressure, just a proactive nudge, matching the survey's explicit recommendation to fix Khanmigo's passivity problem without abandoning rigor. - Notifications/reminders are lower priority –[INFERENCE]' these require a delivery channel (email/push) that doesn't exist in the stack yet and shouldn't block the MVP.

10. Cross-Domain Learning — the Survey’s Named Differentiator

[CODE] This is, concretely, the least-implemented feature in the entire codebase relative to how much weight the reference document puts on it.

- There is no backend pedagogical mode for cross-domain learning. The six modes in pedagogicalEngine.ts are Accelerator, Socratic, Simplifier, Roadmap, Gap Finder, Feynman — “Cross-Domain” is not one of them.
- The only place the words “cross-domain” appear in the frontend is a chat-type toggle inside the already-broken InteractiveAssistant.tsx: switching to “Cross-Domain” builds a system prompt string (“connecting it to unrelated domains... through creative analogies”) — **and that string is never included in the actual /api/generate request that follows.** The toggle changes a local variable that has no downstream effect. A user clicking “Cross-Domain” today gets an identical result to “Focused.”
- No interdisciplinary-example generation, no transferable-skill identification, and no lateral-thinking-challenge generator exist anywhere in the codebase.

Given the survey states this is the one feature “no equivalent anywhere in the market” and the specific reason EdLearn is differentiated at all, **[INFERENCE] this deserves to be treated as a first-class new pedagogical mode, not a chat toggle**, built with the same rigor as the other six modes: a dedicated system prompt (e.g., “explain topic X’s core mechanism using an analogy from field Y, then make the transferable principle explicit”), a defined output schema, a UI renderer, and — ideally — a way to select or randomize the “unrelated domain” for genuine lateral-thinking value rather than a single canned framing.

11. EdQuiz Gap Analysis

[CODE] Assessment does not currently exist as a first-class EdLearn capability. The closest thing is Mode 5 (Gap Finder), which generates 5 MCQs with a “trap index” designed to expose shallow understanding — but per Section 5, this mode is disconnected from the live route, has no UI, and (even if reconnected) has no scoring/persistence layer.

What a complete assessment system needs, per the prompt’s own checklist, mapped against current reality:

Capability	Status [CODE]
Quiz/question generation	[PARTIAL] Prompt logic exists (Gap Finder), disconnected
Difficulty levels	[DONE] difficulty is already a first-class parameter across the AI pipeline
Topic-based quizzes	[PARTIAL] Same as generation — the mechanism exists, isn’t wired up

Capability	Status [CODE]
AI-generated questions	[PARTIAL] Same
Answer evaluation	[MISSING] Not implemented — no route scores a submitted answer
Explanations	[PARTIAL] The Gap Finder schema includes an explanation field per question, unused
Score tracking	[MISSING] Not implemented
Skill mapping	[MISSING] Not implemented
Weak-topic detection	[MISSING] Not implemented
Adaptive quizzes	[MISSING] Not implemented
Progress history	[MISSING] Not implemented for quiz-specific data (general day/topic progress exists, quiz scoring doesn't)
Recommendations from results	[MISSING] Not implemented

How EdQuiz should feed back into EdLearn [INFERENCE]: since EdQuiz is a separate product (not in this repository), the realistic near-term path is (1) EdLearn reconnects and completes its *own* in-app Gap Finder assessment loop first (generate → answer → score → persist → surface a “weak topic” on the dashboard), which is fully within this repo’s control, then (2) define a webhook/API contract so the standalone EdQuiz product can push its own results into the same scoring table once that integration is prioritized by whoever owns EdQuiz.

12. EdCompass Gap Analysis

[CODE] Confirmed not implemented beyond the SSO redirect and Roadmap generation (which is EdLearn’s own feature, not EdCompass’s). Career-goal capture exists (careerGoal field), but none of the following exist: skill-gap analysis feeding a career recommendation, resource-to-career mapping, progress-toward-career-goal tracking, changing skill-relevance signals, or industry-trend-aware roadmap adaptation.

The survey specifically calls out “personalized skill relevance and roadmap generation” as the priority research area for when EdCompass development begins [DOC] — which means, per the survey’s own sequencing, EdCompass-specific work should wait until (a) EdLearn’s own Personalization gap (Section 8) is closed enough to produce a real skill-relevance signal, since EdCompass has nothing to consume otherwise, and (b) the Market-Demand widget’s fabricated “LinkedIn” data (Section 3.1) is replaced with something real enough to inform an actual career recommendation — right now it’s random-jittered mock data and would poison any recommendation built on top of it.

13. EdMentor Gap Analysis

[CODE] Confirmed not implemented beyond the SSO redirect. None of the following exist anywhere in this repository: mentor profiles, mentor discovery/matching, a mentor-side dashboard, a student-side session view, doubt-sharing with a mentor, mentor-student communication, session scheduling/management, or mentor-visible progress. The Hub page’s mentor card is a hardcoded-false placeholder by explicit design (see Section 7).

[INFERENCE] Given EdMentor is a fully separate product outside this repository, EdLearn’s realistic scope here is limited to being a good *data source*: exposing an authenticated API (or webhook payload) that a mentor-side app could call to see a student’s roadmap, completed days, and (once built) knowledge gaps — not building mentor-facing UI inside EdLearn itself.

14. Peer Doubt-Solving

[CODE] Not implemented. [DOC] The survey explicitly frames this as a “planned capability,” not something claimed as already built, and specifically points to threaded-forum schema patterns (Stack Overflow/Discourse-style structures, best-answer marking, reputation) as open research for later development — so this finding matches the survey’s own expectation, unlike the pedagogical-modes gap in Section 5, which contradicts what the survey claims is already working.

Separately and worth flagging: `context.md` (this project’s own internal documentation, not the literature survey) claims a Doubt Forum with upvote badges was completed in “Sprint 2” — that claim does not match the current code at all (no forum routes, no forum models, no forum UI exist). This is a documentation-hygiene issue independent of the literature survey.

Recommendation [INFERENCE]: keep this at the priority the survey itself assigns — a real but lower-tier “planned capability,” sequenced after the pedagogical modes are actually working (Section 5) and personalization exists (Section 8), since a doubt forum without a working learning product behind it has nothing to be “about.”

15. Audio / Voice Gap

[CODE] EdLearn is ahead of the survey’s own expectation here — the survey lists audio/TTS as merely “planned” [DOC], and this repository already has working server-side TTS (`lib/tts.ts`) for both regular study notes and the two-voice Duo Podcast, playable through a dedicated player component.

The caveat, carried forward from the prior full-stack audit: the implementation depends on an **unofficial, keyless Google Translate TTS endpoint** wrapped by the `google-tts-api` package — no SLA, no documented rate limits, and the code’s own comment admits the exact API shape was never verified against a live install. The survey itself flags this exact decision as open research: *“a cost/quality comparison of text-to-speech options — browser-native speech synthesis versus ElevenLabs versus OpenAI TTS — before committing to advanced audio features”* [DOC]. That comparison has not happened; EdLearn committed to the free/unofficial option without it.

Speech-to-text and live voice conversation do not exist at all [CODE] — not promised by the survey, and [INFERENCE] not worth prioritizing until the core pedagogical modes (Section 5) actually work, since a “voice tutor” needs a tutor behavior to speak in the first place.

Recommended roadmap [INFERENCE]: (1) do the TTS provider cost/quality comparison the survey already recommends before scaling audio usage further; (2) do not add STT/voice-conversation before Section 5 is resolved — there’s no functioning conversational mode to attach it to yet.

16. Authentication & User Management

[CODE], carried forward and re-checked against this survey’s specific checklist:

Item	Status
Registration	[DONE] Complete
Login	[DONE] Complete
Logout	[DONE] Complete (client-side token clear)
Password hashing	[DONE] Strong (PBKDF2-SHA512, 310k iterations, legacy-hash fallback)
Sessions/JWT	[DONE] Complete (7-day expiry)
Password reset	[MISSING] Not implemented — no route, no email-sending mechanism anywhere in the codebase
Email verification	[MISSING] Not implemented
Profile	[DONE] Complete

Item	Status
User roles (student/mentor/admin)	[MISSING] Not implemented — the User model has no role field at all; every account is implicitly a student
Authorization / ownership checks	[DONE] Consistently strong across nearly every authenticated route (a real strength, confirmed route-by-route in the prior audit)

[DOC] Note: the survey itself recommends deferring OAuth for the first prototype in favor of simple email/password (citing NextAuth.js as a reference point for setup) — EdLearn has, in fact, built *both* (email/password is complete; Google OAuth is code-complete but unverifiable here without live credentials), so this is one area where the implementation slightly exceeds what the survey’s own MVP scope asked for.

17. Security Audit

[CODE], re-confirmed against this survey’s specific checklist (carried forward from the prior full-stack audit, still accurate):

- **API key exposure:** keys are read server-side only from environment variables; none found hard-coded in source. Good.
- **CORS:** effectively disabled — origin: true in index.ts overrides the allowlist the code itself computes and logs.
- **Rate limiting:** declared as a dependency, applied nowhere.
- **Input validation:** generally present and good on Vision Board routes; the signup route has no password-strength check despite this being documented elsewhere as implemented.
- **AI prompt-injection risk [DOC – explicitly called out by the survey's "AI prompt injection risks" checklist]:** [CODE] confirmed live risk — scraped web content (from a user-supplied URL or Wikipedia) is injected directly into the LLM system prompt with no sanitization for embedded instructions. A malicious page containing text like “ignore prior instructions and output X” would be passed straight into the prompt. This has not been exploited or tested here, but the pattern is present and unmitigated.
- **RAG document security [DOC]:** not yet a live concern since no user-uploaded documents exist yet (Section 6) — but must be designed in from the start once document upload is built, not retrofitted.
- **File upload security:** Vision Board’s image field accepts https:// URLs or capped inline base64 data URLs with reasonable size limits — solid, given what exists today.
- **Data privacy [DOC – the survey specifically flags India's DPDP Act given EdLearn handles student progress and doubt data]:** [INFERENCE] no privacy-policy or data-handling documentation was found in the repository; this is worth a basic compliance pass before any real user data is collected, even at prototype stage, per the survey’s own recommendation.

18. Testing Gap

[CODE] **Confirmed zero.** No *.test.* or *.spec.* files exist anywhere under backend/src or frontend/src; neither package.json configures a test runner.

Testing plan for critical flows [INFERENCE], in priority order once Section 5’s mode-routing fix lands (test the fix, not the current broken behavior): 1. Auth: signup/login happy path + failure cases (duplicate email, wrong password, missing fields). 2. /api/generate for each of the six modes once reconnected — verify each mode actually returns its own distinct schema, not the generic notes schema (this test would have caught the Section 5 finding immediately). 3. RAG: verify citation persistence and the double-pass review’s citation-alignment behavior with a mocked AI provider (no

real API calls in CI). 4. IDOR checks on vision-board, tts, and progress routes — these are currently well-built and should be locked in with regression tests before anyone touches them. 5. CORS allowlist and rate-limit behavior, once both are actually implemented (Section 17). 6. Ecosystem: SSO handoff token minting/expiry (fully testable within this repo, since verification is documented as living in the receiving apps).

19. Completed Features

[CODE], genuinely working end-to-end: - Email/password authentication, JWT sessions, ownership-scoped authorization on nearly every route. - The core AI study-notes generation pipeline: real Groq/Gemini calls, real Wikipedia/URL retrieval injected as grounding context, a genuine second-pass LLM review, and persisted citations. - Duo Podcast: script generation, dual-accent TTS synthesis, dedicated player UI — a fully working feature outside the survey’s original scope. - PDF export of generated notes. - Dashboard summary with Redis caching. - Day-completion tracking and course-completion badges, backed by real database tables. - Vision Board: full CRUD, validated, ownership-scoped — the single most complete feature in the codebase. - The GitHub half of the Market-Demand widget (real GitHub API calls). - SSO handoff minting (the EdLearn-side half of cross-app authentication continuity).

20. Partially Completed Features

[CODE]: - The six pedagogical-mode prompts exist and are well-written but are not reachable from the live UI/API path (Section 5). - RAG grounding is real but shallow — no embeddings/vector search/document upload (Section 6). - News/Media/Books content sections are code-complete integrations (NewsAPI, YouTube Data API, PodcastIndex, Google Books) but every key required to populate them is absent from this repo’s .env, so population is unverified. - Google OAuth is code-complete but unverifiable without live credentials. - The Personalized Roadmap generator works but only as a one-time onboarding step, not an adapting “mode.”

21. Broken Features

[CODE]: - **Cross-Domain chat toggle** — computes a system prompt that is never sent to the API; functionally identical to “Focused” mode regardless of selection. - **AI Tutor chat (InteractiveAssistant.tsx)** — every message re-triggers note generation for the current day’s title (mode 1) instead of answering the user’s actual typed question; the “reply” shown is an unrelated snippet sliced from freshly generated notes. - **CORS** — the computed origin allowlist is dead code; origin: true accepts every origin. - **Kubernetes backend deployment** — will crash-loop as configured; required secrets are never injected via the manifests.

22. Mock / Placeholder Features

[CODE]: - **Market-Demand “LinkedIn” data** — a hardcoded array with Math.random() jitter, not scraped, despite implying live market data. - **Hub page mentor/compass cards** — sample objects gated behind hardcoded false flags; honestly shows empty states rather than fake data, but the underlying feature doesn’t exist. - **Rate limiting and signup password-strength enforcement** — documented elsewhere (context.md) as implemented; confirmed absent from the running code.

23. Feature Gaps (Master List)

#	Missing/Incomplete Feature	Why Needed	Current State	Priority	Owner
1	Wire /api/generate to actually call getPedagogicalModeConfig() per mode, for modes 1, 2, 3, 5, 6	This is the survey's core thesis; currently all modes return identical generic notes	Prompts exist, disconnected	P0	Member 1
2	Re-enable the mode-selector UI + build renderers for each mode's distinct schema	No user can currently select or view a differentiated mode	UI explicitly disabled with "modes suspended" copy	P0	Member 3
3	Build Cross-Domain Learning as a real seventh (or first-class) pedagogical mode	The survey's named, unmatched differentiator; currently a dead toggle	5% implemented	P0	Member 1
4	Persist Gap Finder results into a Knowledge-Gap/mastery table	Required for the survey's "individual skill-relevance signal" claim	No persistence layer exists	P0	Member 4
5	Fix the AI Tutor chat to use the user's real message	Currently the most misleading working feature in the product	Regenerates unrelated notes instead of answering	P0	Member 1 + Member 4
6	Fix CORS (origin: true → real allowlist)	Currently any origin can make credentialed requests	Dead allowlist code	P0	Member 4
7	Apply rate limiting to auth + AI-cost routes	Zero abuse/cost protection today	Dependency installed, unused	P0	Member 4
8	Wire real secrets into the K8s backend deployment	Backend cannot currently boot in a real cluster	Secrets never injected	P0	Member 5
9	Enforce signup password strength server-side	Documented as done, absent from code	No length/composition check	P0	Member 4

#	Missing/Incomplete Feature	Why Needed	Current State	Priority	Owner
10	Basic chunking + embeddings + vector search for RAG	Needed to move RAG from “keyword search” toward what the survey implies	No embeddings/vector store anywhere	P1	Member 2
11	User-uploaded document ingestion for RAG	Survey explicitly frames a personal-knowledge-base RAG use case	Not implemented	P1	Member 2
12	Reasoning-verification safeguard for STEM/code content	Survey explicitly warns RAG alone isn’t enough for reasoning-heavy tasks	Only a ground-ing/style double-pass exists	P1	Member 2
13	Basic personalization: skill mastery + adaptive next-topic recommendation	Second named differentiator in the survey	Static onboarding profile only	P1	Member 4
14	Complete EdQuiz-equivalent assessment loop inside EdLearn (score, persist, surface weak topics)	Required before any real EdQuiz integration is meaningful	Gap Finder mode disconnected, no scoring	P1	Member 1 + Member 4
15	Basic engagement layer: daily-goal surface, streaks	Survey explicitly cites Khanmigo’s 15% usage as the risk of skipping this	Not implemented	P1	Member 3
16	Replace fabricated “LinkedIn” market-demand data	Currently fake and would poison any future EdCompass recommendation built on it	Hardcoded array + jitter	P1	Member 2
17	TTS provider cost/quality comparison + possible migration	Survey explicitly recommends this before scaling audio	Running on an unofficial free endpoint	P2	Member 2

#	Missing/Incomplete Feature	Why Needed	Current State	Priority	Owner
18	Password reset + email verification	Standard auth completeness gap	Not implemented	P2	Member 4
19	User roles (student/mentor/teacher)	Needed before any mentor-facing surface can exist	No role field in schema	P2	Member 4
20	Define an API contract for EdQuiz/EdCompass/EdMentor to push data back to EdLearn	Required to ever close the loop the survey describes	Only one-way SSO redirect exists	P2	Member 4
21	Peer doubt-solving forum	Survey itself marks this as a future “planned capability,” not urgent	Not implemented	P3	Member 5
22	Basic privacy/data-handling documentation (DPDP awareness)	Survey explicitly flags this given student progress/doubt data	Not documented	P3	Member 5
23	Speech-to-text / live voice conversation	Not promised by the survey; needs a working conversational mode first	Not implemented	P3	Member 2
24	Automated test suite across all of the above	No regression protection for any fix in this list	Zero tests exist	P1 (ongoing)	Member 5

24. Research-Driven Gaps From Literature Survey

Direct mapping of the survey’s four stated market gaps [DOC] to EdLearn’s actual current state [CODE]:

1. **“No competitor combines rigorous answer-withholding teaching with an integrated assessment → roadmap → mentorship pipeline.”** EdLearn does not currently close this gap either — the answer-withholding modes aren’t reachable (Section 5), and the pipeline to EdQuiz/EdCompass/EdMentor is a one-way redirect with no shared context (Section 7). The gap the survey identifies in the market is, right now, also present inside EdLearn itself.
2. **“No competitor tracks an individual learner’s changing skill relevance.”** Not implemented (Section 8) — no mastery tracking, no gap persistence, no adapting roadmap.
3. **“Rigor and engagement rarely coexist.”** EdLearn currently has neither in a working state: rigor (Socratic/Feynman) is disconnected, and engagement (streaks/goals/proactive nudges) doesn’t exist (Section 9).
4. **“Cross-domain/lateral learning has no equivalent in the market.”** This is true of the market, and — per Section 10 — currently also true of EdLearn’s own shipped product, since the feature is

a non-functional toggle.

Honest framing for whoever reads this alongside the survey [INFERENCE]: the survey is a strong, well-researched case for *why* EdLearn’s planned design would be differentiated if built. It should not be read as evidence that the differentiation already exists in the current code — it doesn’t yet, on any of its four named axes.

25. Features That Must Be Added

Only features that make sense given both the actual codebase state and the reference document — not a generic feature dump:

Core AI Learning [P0]: reconnect the six modes to `/api/generate`; build Cross-Domain Learning as a real mode; fix the AI Tutor chat to use real conversation input.

RAG [P1]: chunking + embeddings + basic vector search; user document upload; a reasoning-verification pass for STEM content; retrieval-relevance filtering before prompt injection.

Assessment [P1]: complete EdLearn’s own Gap Finder scoring/persistence loop before attempting any real EdQuiz data contract.

Personalization [P1]: a mastery/knowledge-gap table; a basic next-topic recommender reading from it.

Ecosystem [P2]: a defined push-back API contract for EdQuiz/EdCompass/EdMentor — cannot be fully built until those sibling teams are ready, but EdLearn’s receiving side should exist and be tested with mock payloads.

Engagement [P1, scoped down deliberately]: a daily-goal surface and a streak counter — explicitly *not* heavier gamification, per the survey’s own South Korea caution against forcing adoption.

Community [P3]: peer doubt-solving forum — correctly scoped as future work by the survey itself; not recommended before P0/P1 items land.

Voice [P2/P3]: a TTS provider comparison/migration (P2, since current TTS already works but is fragile); STT/voice-conversation deliberately **not** recommended yet (P3) — [INFERENCE] there’s no working conversational mode to attach it to until Section 5 is fixed.

Explicitly not recommended right now [INFERENCE]: full gamification/badges-for-everything systems, a heavyweight vector-database migration (Pinecone-scale) before the far more basic ingestion pipeline exists, and any new mentor-facing UI inside EdLearn itself (that belongs in the EdMentor product, per Section 13).

26. MVP Features

MVP — MUST COMPLETE (demonstrates the actual differentiator the survey describes: *AI-guided learning + grounded knowledge + personalization + assessment + progression toward roadmap/mentorship*): - All six pedagogical modes actually reachable and behaviorally distinct (Section 5) — this alone is the single biggest lift and the one non-negotiable item, since without it there is no “AI-guided learning” to demonstrate. - Cross-Domain Learning built as a real mode, not a toggle. - RAG grounding kept at its current honest level (real retrieval + citations) — full embeddings/vector search is **not** required for MVP, just for production-readiness (Section 6). - Gap Finder’s results persisted and surfaced back to the learner (minimal personalization loop). - CORS fixed, rate limiting applied, K8s secrets wired — an MVP that can’t be safely demoed or deployed isn’t an MVP. - AI Tutor chat fixed to actually converse.

POST-MVP — SHOULD COMPLETE: - Embeddings/vector search and document upload for RAG. - Basic engagement layer (daily goal, streak). - Password reset, email verification, user roles. - Replace the fabricated market-demand data. - A defined (even if unused-by-partners-yet) API contract for EdQuiz/EdCompass/EdMentor push-back.

FUTURE — OPTIONAL: - Peer doubt-solving forum. - Speech-to-text / live voice tutor. - TTS provider migration to a paid/higher-SLA option. - Full adaptive-difficulty and recommendation engine.

[INFERENCE] Advanced ecosystem and community features should not be allowed to delay the MVP — none of them are reachable in a way that matters until the six modes and Cross-Domain Learning actually work, since those are what every downstream feature (assessment, personalization, ecosystem) depends on having real data from.

27. P0/P1/P2/P3 Priority Backlog

See the consolidated Feature Gap List in Section 23 — it is already fully prioritized P0 through P3 with owners assigned. (Repeating it here would duplicate the table; refer to Section 23 as the single source of truth for the backlog.)

28. 5-Member Responsibility Matrix

The prompt’s suggested structure fits this codebase well; the biggest adaptation is that **Member 4’s “Backend, Database & Ecosystem” role absorbs the security/CORS/rate-limiting/K8s fixes**, since those are backend-adjacent and there’s no clean way to split “backend” from “backend security” across two different people without constant collisions in `index.ts`. Member 5 focuses on testing, DevOps, and the lower-priority community/security-review work.

Area	Member 1 (AI Learning Engine)	Member 2 (RAG & Knowledge System)	Member 3 (Frontend & Learning Experience)	Member 4 (Backend, Database & Ecosystem)	Member 5 (Testing, DevOps, Security & Community)
Six pedagogical modes	Owns prompt-to-route wiring	—	Owns mode-selector UI + per-mode renderers	Reviews route changes	Tests each mode’s distinct output
Cross-Domain Learning	Owns new mode design	—	Owns UI	—	Tests
AI Tutor chat fix	Owns new contract	—	Owns UI rebuild	Owns endpoint	Tests
RAG upgrade	—	Owns chunking/embeddings/vector search/doc upload	—	Supports schema changes	Tests retrieval quality
Personalization / Gap Finder persistence	Owns Gap Finder reconnection	—	Owns results UI	Owns schema + persistence	Tests
Security (CORS/rate limit/K8s secrets)	—	—	—	Owns	Verifies with scripted tests
Ecosystem contract (EdQuiz/EdCompass/EdMentor)	—	—	Owns Hub UI once real data exists	Owns API contract	Tests with mock payloads
Engagement (daily goal/streak)	—	—	Owns	Owns data source	Tests

Area	Member 1 (AI Learning Engine)	Member 2 (RAG & Knowledge System)	Member 3 (Frontend & Learning Experience)	Member 4 (Backend, Database & Ecosystem)	Member 5 (Testing, DevOps, Security & Community)
Market-demand data fix	—	Owns	—	—	—
Testing foundation	Writes tests for own area	Writes tests for own area	Writes tests for own area	Writes tests for own area	Owns the suite + CI
Doubt forum (future)	—	—	Owns UI (when scheduled)	Owns schema/routes (when scheduled)	Owns moderation approach
Documentation reconciliation	—	—	—	—	Owns

29. Detailed Member 1 Plan — AI Learning Engine

Primary Responsibility: Making the six pedagogical modes, and Cross-Domain Learning, actually reachable and behaviorally distinct — this is the highest-value work in the entire plan, since it’s the survey’s core thesis.

Current Work Already Completed [CODE]: all six mode prompts are written and schema-designed in `pedagogicalEngine.ts`; the Duo Podcast mode (7) is fully working end-to-end and can serve as a reference pattern for “a mode that’s properly wired up.”

Feature Gaps: modes 1/2/3/5/6 are disconnected from `/api/generate`; Cross-Domain Learning doesn’t exist as a mode at all; the AI Tutor chat doesn’t use real conversational input.

New Features To Implement: - Branch `/api/generate` by `modeNumber` to call `getPedagogicalModeConfig(modeNumber, ...)` for modes 1, 2, 3, 5, 6 (following the exact pattern mode 7 already uses), instead of the current single fixed prompt. - Design and implement Cross-Domain Learning as mode 8 (or renumber thoughtfully): a real system prompt that pairs the topic with a genuinely different field and makes the transferable principle explicit, with its own output schema. - Design the real conversational contract for the AI Tutor chat (the user’s actual typed message must reach the model, with `currentDay.title` as context, not as the sole input).

Exact Tasks: 1. Refactor `/api/generate`’s non-podcast branch to route through `getPedagogicalModeConfig()` per mode number. 2. Add response-shape handling per mode (each mode’s JSON schema differs from the generic notes schema) so the API response for modes 1/2/3/5/6 carries its own structure, the way mode 7 already returns `script` instead of `contentBlocks`. 3. Design and prompt-engineer the Cross-Domain Learning mode. 4. Design the Tutor’s real request/response contract and hand the endpoint spec to Member 4.

Files/Directories: `backend/src/lib/ai/pedagogicalEngine.ts`, `backend/src/index.ts` (`/api/generate` route body).

Dependencies: Needs Member 3 to build a renderer for each newly-reachable schema (a wired-up Socratic mode with no UI to show its `trapOptions` is still not usable); needs Member 4 to implement whatever new Tutor endpoint this work specifies.

Expected Deliverables: all six original modes distinguishable in a real API response; a working Cross-Domain Learning mode; a documented Tutor contract.

Definition of Done: calling `/api/generate` with each of modes 1–6 returns that mode’s own distinct JSON schema, verified by an automated test (Member 5) that would fail if two different modes ever returned identical output again.

Priority: P0

Estimated Effort: 6–8 days

30. Detailed Member 2 Plan — RAG & Knowledge System

Primary Responsibility: Moving RAG from “real but shallow” to production-ready, and fixing the one clearly fabricated data source in the app.

Current Work Already Completed [CODE]: genuine Wikipedia/URL retrieval, genuine citation persistence, a working (if style-focused, not reasoning-focused) double-pass review.

Feature Gaps: no chunking, embeddings, vector store, document upload, personal knowledge base, or retrieval evaluation; no reasoning-verification safeguard for STEM content; the Market-Demand widget’s “LinkedIn” data is fabricated.

New Features To Implement: - A chunking + embedding + vector-search layer, using pgvector on the existing Postgres instance as the pragmatic first choice (matches the survey’s own open research question about pgvector vs. Pinecone [DOC], and avoids introducing a whole new datastore for a prototype). - User-uploaded document ingestion (PDF/notes) as a new RAG source, feeding the same vector-search layer. - A lightweight relevance-check step before injecting any retrieved content into a prompt. - A distinct reasoning-verification pass for STEM/code-heavy topics, separate from the existing citation/style double-pass review. - Replace the hardcoded “LinkedIn” market-demand array with either a real, honestly-scoped data source or clearly relabeled synthetic baseline data.

Exact Tasks: 1. Add pgvector extension + an Embedding/DocumentChunk table to schema.prisma. 2. Build ingestion: chunk → embed → store, for both scraped web content and (once upload exists) user documents. 3. Replace the current “top-2 Wikipedia search results” retrieval with a similarity search over the new vector store, falling back to the existing keyword approach when no embeddings exist yet for a topic. 4. Add a reasoning-verification prompt pass for STEM/code content specifically. 5. Rework cronScraper.ts’s DEFAULT_TRENDS fabrication into either a real signal or explicit synthetic-baseline labeling.

Files/Directories: backend/src/lib/scrapper.ts, backend/prisma/schema.prisma, new backend/src/lib/rag/**, backend/src/lib/cronScraper.ts.

Dependencies: Needs Member 4 for schema-migration review; needs a product decision (owner: whoever manages the roadmap) on whether document upload is in scope for this phase or deferred.

Expected Deliverables: a working embedding-based retrieval path; document upload (post-MVP); an honestly-labeled or genuinely real market-demand data source.

Definition of Done: a query against a previously-scraped topic returns results via vector similarity, not just keyword search; the market-demand widget no longer silently fabricates a data source.

Priority: P1 (embeddings/vector search), P1 (market-demand fix)

Estimated Effort: 8–10 days

31. Detailed Member 3 Plan — Frontend & Learning Experience

Primary Responsibility: Making the reconnected pedagogical modes, Cross-Domain Learning, and a basic engagement layer actually visible and usable to a student — and fixing the UI’s currently-broken/misleading surfaces.

Current Work Already Completed [CODE]: the generic notes renderer (LivingDocument.tsx) is polished and complete; the Duo Podcast player is a good reference pattern for a mode-specific UI; Vision Board, dashboard, and onboarding flows are solid.

Feature Gaps: the mode-selector UI is explicitly disabled (“modes suspended”); there is no renderer for any of the five reconnected modes’ distinct schemas; the AI Tutor chat UI needs to be rebuilt once Member 1/4 finish its contract; no engagement surface exists.

New Features To Implement: - Re-enable the mode-selector buttons in `LeftNavigationPanel.tsx` (remove the “suspended” copy once modes 1–6 are actually wired up by Member 1). - Build a distinct UI renderer per mode: an interactive scenario/trap-option flow for Socratic, a keystone-question quiz UI for the Simplifier and Gap Finder, an adversarial explain-back UI for Feynman, and a dedicated interdisciplinary-example layout for Cross-Domain Learning. - Rebuild the AI Tutor chat once Member 1/4’s real contract is ready. - Build a daily-goal / “continue learning” surface and a streak counter on the dashboard.

Exact Tasks: 1. Remove the “modes suspended” disclaimer and re-render all six mode buttons, gated on Member 1’s backend work landing first. 2. Build one new renderer component per distinct mode schema (5 new components, roughly). 3. Rebuild `InteractiveAssistant.tsx` against the new Tutor contract. 4. Add a “Today” widget to the dashboard summarizing the next suggested step + a streak counter.

Files/Directories: `frontend/src/components/Layout/LeftNavigationPanel.tsx`, `frontend/src/component` (new renderer components), `frontend/src/components/Layout/InteractiveAssistant.tsx`, `frontend/src/app/dashboard/page.tsx`.

Dependencies: Blocked on Member 1’s mode-wiring work for the renderers; blocked on Member 1+4’s Tutor contract for the chat rebuild; can start the engagement widget independently.

Expected Deliverables: all six modes selectable and visibly distinct; a working Tutor chat; a basic engagement surface.

Definition of Done: a student can select each of the six modes and see genuinely different content/interaction, not the same notes template six times.

Priority: P0 (modes + Tutor), P1 (engagement)

Estimated Effort: 8–10 days

32. Detailed Member 4 Plan — Backend, Database & Ecosystem

Primary Responsibility: Data model changes needed for personalization/assessment, the security fixes that block a safe deployment, and defining (not necessarily fully building, since partners aren’t ready) the ecosystem data contract.

Current Work Already Completed [CODE]: solid API architecture, strong ownership/IDOR checks, working SSO handoff minting, a coherent (if incomplete) Prisma schema.

Feature Gaps: CORS bypass, missing rate limiting, non-bootable K8s secrets, missing password-strength enforcement, no knowledge-gap/mastery persistence, no ecosystem push-back contract, no user roles.

New Features To Implement: - Fix CORS and apply rate limiting (carried forward from the prior full-stack audit — still unresolved). - Wire K8s secrets properly. - Enforce password strength server-side. - A KnowledgeGap/mastery table and the scoring endpoint Gap Finder needs once Member 1 reconnects it. - A role field on User (student/mentor/admin) to unblock any future mentor-facing work. - A defined (initially mock-tested) API contract for EdQuiz/EdCompass/EdMentor to push results back into EdLearn.

Exact Tasks: 1. Replace `origin: true` with the existing `allowedOrigins allowlist` logic in `index.ts`. 2. Apply `express-rate-limit` to `/auth/signup`, `/auth/login`, `/generate`, `/facts`, `/assistant/classify`. 3. Create a K8s Secret and reference it via `envFrom.secretRef` in `k8s/backend/deployment.yaml`. 4. Add server-side password-strength validation to `/api/auth/signup`. 5. Design and implement the Gap Finder scoring/persistence endpoint (POST `/api/gap-finder/submit` or similar) once Member 1’s mode is reconnected. 6. Add role to the User model with a safe default migration. 7. Draft (and stub-test with mock payloads) the ecosystem push-back contract.

Files/Directories: backend/src/index.ts, backend/prisma/schema.prisma, k8s/**.

Dependencies: Needs Member 1's mode reconnection before the Gap Finder scoring endpoint has anything real to score; the ecosystem contract's real integration depends on sibling-app teams outside this repo, so this piece is design-and-stub only for now.

Expected Deliverables: a safely deployable backend; a knowledge-gap persistence layer; a documented, testable ecosystem contract stub.

Definition of Done: CORS/rate-limit/K8s items pass Member 5's scripted verification; a Gap Finder submission is persisted and readable by the dashboard.

Priority: P0 (security items), P1 (Gap Finder persistence, roles, ecosystem contract)

Estimated Effort: 7-9 days

33. Detailed Member 5 Plan — Testing, DevOps, Security & Community

Primary Responsibility: Building the test suite this project has never had, verifying every other member's fix, and owning the lower-priority community/documentation work.

Current Work Already Completed [CODE]: none directly attributable — this is a net-new function for the project (zero tests currently exist).

Feature Gaps: the entire testing category; documentation drift between context.md/README and the actual code (independent of this survey, carried from the prior audit); the peer doubt-solving forum (survey-acknowledged future work).

New Features To Implement: - A test runner for both backend and frontend. - Regression tests written alongside each other member's P0/P1 fix, not after the fact. - A basic data-privacy/DPDP-awareness note for the documentation, per the survey's explicit flag. - The peer doubt-solving forum, scheduled last, matching the survey's own "planned capability" (not urgent) framing.

Exact Tasks: 1. Stand up Vitest/Jest (backend) and Vitest + React Testing Library (frontend). 2. Write the mode-distinctness test described in Member 1's Definition of Done — this single test is the most valuable regression guard in the whole plan, since it directly encodes "the six modes must actually differ." 3. Write CORS/rate-limit verification scripts against Member 4's fixes. 4. Run the full Docker/K8s smoke test once Member 4's secret-wiring lands. 5. Reconcile README.md/context.md against the merged code (forum claims, rate-limiting claims, password-strength claims). 6. Add a short data-privacy/DPDP-awareness section to the documentation. 7. Once P0/P1 work across the team is stable, design and build the peer doubt-solving forum (schema, routes, basic UI, minimal AI moderation).

Files/Directories: new backend/src/**/*test.ts, new frontend/src/**/*test.tsx, README.md, context.md, k8s/** (verification only), new backend/src/routes/forum.ts (later phase).

Dependencies: Every regression test in this plan is blocked on the corresponding fix landing first; the K8s smoke test is blocked on Member 4; the forum is deliberately sequenced last.

Expected Deliverables: a running, meaningful test suite; a verified, safely deployable stack; accurate documentation; the doubt forum as a final-phase addition.

Definition of Done: the test suite passes and specifically catches the class of bug this audit found (modes returning identical output); a fresh clone + documented setup produces a working stack; docs match code.

Priority: P1 (test foundation, ongoing throughout), P3 (forum)

Estimated Effort: 6-8 days initial test foundation, then ongoing verification work through the whole plan, then 3-5 days for the forum at the end

34. Dependencies

BLOCKING TASKS (must happen first): - Member 1's /api/generate mode-routing fix blocks Member 3's per-mode UI renderers and Member 4's Gap Finder scoring endpoint. - Member 1+4's AI Tutor contract blocks Member 3's Tutor UI rebuild. - Member 4's security fixes (CORS/rate-limit/K8s secrets) block Member 5's deployment smoke test. - Member 2's embeddings/vector-search layer blocks any future retrieval-evaluation testing.

PARALLEL TASKS (can run simultaneously from day one): - Member 4's CORS fix, rate-limit fix, and K8s secret wiring (fully independent of the AI-mode work). - Member 2's market-demand data fix (independent of the RAG/embeddings work). - Member 3's engagement-widget build (independent of the mode-reconnection work, since it only needs existing Progress data). - Member 5's test-runner scaffolding (no code dependency).

INTEGRATION TASKS (require multiple members): - Six-modes end-to-end: Member 1 (backend wiring) + Member 3 (UI renderers) + Member 5 (regression test). - AI Tutor fix: Member 1 (contract) + Member 4 (endpoint) + Member 3 (UI). - Gap Finder persistence: Member 1 (mode reconnection) + Member 4 (schema + scoring endpoint) + Member 3 (results UI). - Ecosystem contract: Member 4 (contract design) + Member 5 (mock-payload testing).

Database (KnowledgeGap/mastery schema, role field)

↓

Backend mode-routing fix + security fixes (can run in parallel with each other)

↓

AI Learning Engine (modes reconnected, Cross-Domain built, Tutor contract defined)

↓

Frontend Integration (mode renderers, Tutor rebuild, engagement widget)

↓

RAG upgrade (can start in parallel once schema work begins, doesn't block the modes fix)

↓

End-to-End Testing (validates modes are actually distinct, security holds, K8s boots)

↓

Deployment

35. Parallel Work Plan

See Section 34's "PARALLEL TASKS" — restated as a week-one starting lineup: Member 4 starts security fixes immediately; Member 2 starts the market-demand fix and begins embeddings-layer design; Member 3 starts the engagement widget; Member 5 scaffolds the test runner. Member 1 starts the mode-routing refactor immediately as well — it has no code dependency on anyone else, only downstream consumers.

36. Integration Plan

1. **First to complete:** Member 4's security fixes (no dependents block their *start*) and Member 1's mode-routing refactor (the critical path for everything downstream).
2. **Database changes merge:** the KnowledgeGap/mastery table and role field should merge early, once Member 1's mode work confirms exactly what a Gap Finder submission needs to store.
3. **Backend APIs available:** the six-mode /api/generate behavior and the Gap Finder scoring endpoint should be manually verified by Member 1+4 before Member 3 builds UI against them.
4. **Frontend connects to APIs:** immediately once each backend piece lands, following the codebase's existing per-page axios pattern.
5. **AI integration:** the mode-routing fix and Cross-Domain Learning build are the two pieces of *new* AI integration work in this plan; both should be functionally verified (via manual testing, then Member 5's automated test) before frontend UI work is considered final.

6. **Complete-system testing begins:** once the six-mode UI + Tutor rebuild both land — that’s the last major moving piece.
7. **Feature freeze:** after the MVP list in Section 26 is complete; POST-MVP and FUTURE items are explicitly out of scope for the first freeze.

Final integration checklist: - [] Each of the six modes returns a verifiably distinct schema from /api/generate - [] Cross-Domain Learning is a real, working mode - [] Gap Finder results are persisted and visible somewhere in the UI - [] AI Tutor chat answers the literal question typed - [] CORS allowlist and rate limiting verified with scripted tests - [] `kubectl apply -f k8s/` produces a running backend using real Secrets - [] README/context.md reconciled against the merged code - [] Full manual E2E pass across signup → onboarding → all six modes → Cross-Domain → Gap Finder → dashboard → vision-board → hub

37. Timeline

Week	Member 1 (AI Learning)	Member 2 (RAG & Knowledge)	Member 3 (Frontend)	Member 4 (Backend/Ecosystem)	Member 5 (Test/DevOps)
1	Refactor /api/generate mode-routing (P0, critical path)	Market-demand data fix; begin embeddings-layer design	Engagement widget (independent); prep mode-renderer component shells	Security fixes: CORS, rate limiting, K8s secrets; password-strength enforcement	Scaffold test runner; write security regression tests as Member 4 lands each fix
2	Design + implement Cross-Domain Learning mode; design AI Tutor contract	Begin pgvector schema + chunking pipeline	Build per-mode UI renderers against Member 1’s now-reconnected modes	Implement Gap Finder scoring/persistence schema + endpoint; add role field	Write the mode-distinctness regression test; IDOR regression suite
3	Support Member 4 on Tutor endpoint implementation	Continue embeddings/vector search build	Rebuild AI Tutor chat UI; wire Gap Finder results UI	Implement AI Tutor endpoint; draft ecosystem push-back contract	Test Tutor endpoint; begin K8s smoke test
4	Polish + bug-fix modes/Cross-Domain based on QA feedback	Finish embeddings layer; document upload if in scope	Final polish on all six mode UIs + engagement widget	Bug-fix buffer; finalize ecosystem contract stub	Full manual E2E pass; K8s + Docker smoke test
5 (buffer)	Cross-review, bug-fix buffer	Cross-review, bug-fix buffer	Cross-review, bug-fix buffer	Cross-review, bug-fix buffer	Documentation reconciliation; final MVP sign-off

[INFERENCE] No fixed deadline was supplied, so this is a 5-week estimate; the six-modes reconnection (Member 1, Week 1) is the true critical path — everything else in the plan either depends on it or can proceed in parallel but shouldn’t be considered “MVP-complete” until it lands.

38. Git / Branch Strategy

main

— fix/pedagogical-mode-routing	(Member 1 – the critical-path fix)
— feature/cross-domain-learning	(Member 1, after the above)
— feature/ai-tutor-rebuild	(Member 1 designs contract → Member 4 implements → Member 3 wires)
— feature/rag-embeddings	(Member 2)
— fix/security-hardening	(Member 4 – CORS, rate limiting, K8s secrets)
— feature/gap-finder-persistence	(Member 4 schema/endpoint → Member 3 UI)
— feature/engagement-widget	(Member 3)
— chore/market-demand-honesty	(Member 2)
— test/regression-suite	(Member 5, grows continuously)
— feature/doubt-forum	(Member 5, scheduled last)

- **File ownership:** backend/src/index.ts is the highest-collision file (touched by Members 1 and 4); land small, frequent PRs and rebase daily.
- **Commit strategy:** one purpose per commit, referencing the specific gap it closes (e.g., “fix: route /api/generate through getPedagogicalModeConfig per mode — closes mode-routing finding”).
- **PR strategy:** fix/pedagogical-mode-routing requires review from Member 3 (since it determines the API shape their renderers depend on) before merge. feature/ai-tutor-rebuild requires sign-off from Members 1, 3, and 4 together, since it spans all three.
- **Merge order:** security fixes and mode-routing merge independently and immediately once reviewed; feature/ai-tutor-rebuild and feature/gap-finder-persistence merge only once every sequential piece is complete and Member 5 has a passing regression test.
- **Conflict prevention:** nobody touches backend/src/index.ts without pulling latest first; schema changes (Member 2’s embeddings tables, Member 4’s KnowledgeGap/role fields) go through a single reviewer (Member 4) to avoid two people writing conflicting Prisma migrations in the same week.

39. Testing Plan

See Section 18 for the full critical-flow list. Summary of what must exist before this project can be called “tested” at even a baseline level: 1. Auth flow tests (signup/login happy + failure paths). 2. The mode-distinctness test (each of the six modes returns its own schema) — the single highest-value test in this entire plan, since it directly encodes the survey’s core thesis as an automated check. 3. RAG/citation persistence tests with a mocked AI provider. 4. IDOR regression tests on vision-board/tts/progress routes. 5. CORS allowlist + rate-limit behavior tests. 6. Gap Finder scoring/persistence tests. 7. SSO handoff token minting/expiry tests. 8. A Docker + Kubernetes smoke test confirming the full stack boots from a clean environment.

40. Security Plan

Priority-ordered, consolidating Section 17’s findings into actions: 1. Fix CORS (origin: true → real allowlist) — **P0**. 2. Apply rate limiting to auth + AI-cost routes — **P0**. 3. Wire real K8s Secrets for all sensitive env vars — **P0**. 4. Enforce signup password strength server-side — **P0**. 5. Add basic prompt-injection awareness to the RAG pipeline (at minimum, strip/flag obvious instruction-like patterns in scraped content before injecting it into a system prompt) — **P1**, since this is currently a live, unmitigated pattern. 6. Design RAG document security *before* user-upload ingestion ships (Member 2), not after — **P1**. 7. Add password reset + email verification — **P2**. 8. Add a basic DPDP-awareness note to documentation given student progress/doubt data is already being handled — **P3**, but should not be skipped entirely.

41. Deployment Plan

1. Fix K8s secret injection (Member 4) — currently the single blocker to any real cluster deployment.
 2. Full Docker Compose smoke test (both Mode A and Mode B per the README) — Member 5.
 3. Full `kubectl apply -f k8s/` smoke test against the fixed manifests — Member 5.
 4. Confirm production build succeeds for both frontend and backend (`npm run build` in each) with no new secrets or config assumptions introduced by this plan's changes.
 5. Document the verified deployment steps, replacing any stale instructions in `README.md`.
-

42. Final Definition of Done

EdLearn can only be declared complete against this literature survey when:

- All six pedagogical modes are reachable in the live product and demonstrably produce distinct behavior, not the same generic notes output.
- Cross-Domain Learning is a real, working mode, not a dead UI toggle.
- AI responses are properly grounded where RAG is required, with citations intact.
- A reasoning-verification safeguard exists for STEM/code-heavy content, distinct from the current style/grounding double-pass.
- Learner progress, including knowledge gaps (from Gap Finder), is persisted and actually feeds back into what the learner sees next.
- Assessment (at minimum, EdLearn's own Gap Finder loop) works end-to-end: generate → answer → score → persist → surface.
- The ecosystem contract to EdQuiz/EdCompass/EdMentor is at least defined and testable with mock payloads, even if the sibling apps aren't ready to consume it yet.
- Authentication works, including the currently-missing password-strength enforcement.
- CORS is genuinely restricted and rate limiting is genuinely applied.
- No feature described in the README, `context.md`, or presented to a user implies functionality that isn't actually there (the Cross-Domain toggle, the AI Tutor chat, and the LinkedIn market-demand data are the three concrete violations of this principle found in this audit).
- The mode-distinctness regression test (Section 39) and the core auth/IDOR/security tests pass.
- Production build and deployment (Docker + Kubernetes) both work from a clean environment.
- Documentation accurately reflects the shipped code.

EXACT ORDER TO START DEVELOPMENT

1. Member 1 begins refactoring `/api/generate` to route through `getPedagogicalModeConfig()` per mode — this is the single highest-leverage fix in the whole plan and blocks the most downstream work.
2. Member 4 begins the CORS fix, rate limiting, and K8s secret wiring in parallel — fully independent, no reason to wait.
3. Member 2 begins the market-demand data fix and starts designing the embeddings/chunking schema in parallel — also independent.
4. Member 3 begins the engagement widget (daily goal/streak) in parallel, using data that already exists in Progress.
5. Member 5 scaffolds the test runner and writes the mode-distinctness test *as soon as* Member 1's first mode (e.g., Socratic) is reconnected — don't wait for all six.
6. Once Member 1's mode-routing fix lands, Member 3 builds the per-mode UI renderers against the now-real API responses.
7. Member 1 designs and builds Cross-Domain Learning as a genuine new mode, following the same pattern just proven out on modes 1-6.
8. Member 1 and Member 4 jointly design the AI Tutor's real conversational contract; Member 4 implements the endpoint; Member 3 rebuilds the chat UI against it.
9. Member 4 implements the Gap Finder scoring/persistence schema and endpoint once Member 1's Gap Finder mode is live; Member 3 builds the results UI.
10. Member 2 finishes the embeddings/vector-search layer and (if in scope for this phase) document upload.

11. Member 4 drafts the EdQuiz/EdCompass/EdMentor push-back API contract; Member 5 tests it with mock payloads.
12. Member 5 runs the full Docker + Kubernetes smoke test against Member 4's security/deployment fixes.
13. Whole team runs a full manual end-to-end pass against the MVP checklist (Section 26).
14. Member 5 reconciles README/context.md against the final merged code.
15. Final deployment.